5. BÚSQUEDA LOCAL
(PARTE 2)

IA 3.2 - Programación III

1° C - 2026

Dr. Mauro Lucci

🔙 Repaso

La ascensión de colinas se **atasca** en picos y mesetas (máximo local plano o terraza).

Los pasos laterales a veces permiten **escapar** de una terraza.

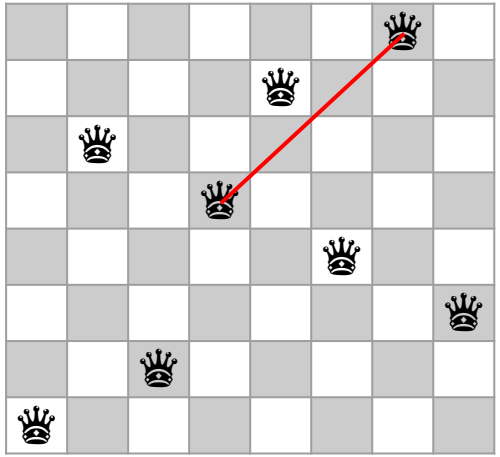
¿Cómo **escapar** en los demás casos? 🤔

Motivación

📖 Ejemplo – 8 Reinas

Supongamos que en la aplicación de la ascensión de colinas al problema de las 8-reinas llegamos al siguiente estado:

…



f(8,3,7,4,2,5,1,6) = 1

A continuación, ¿a qué sucesor nos movemos? 🤔

¡Hagamos las cuentas!…

📖 Ejemplo – 8 Reinas

Supongamos que en la aplicación de la ascensión de colinas al problema de las 8-reinas llegamos al siguiente estado:

…

3	3	3	3	2	3	🙁	3
3	3	4	2	🙁	4	2	4
2	🙁	3	3	5	4	2	3
3	2	4	🙁	4	4	3	2
3	3	4	3	4	🙁	2	3
3	5	3	2	4	3	2	🙁
4	3	🙁	2	2	3	3	3
🙁	3	3	2	2	3	2	3

f(8,3,7,4,2,5,1,6) = 1

El menor valor objetivo entre los sucesores es 2 > 1. Estamos en un mínimo local que no es global, donde todos los sucesores tienen peor valor objetivo. En el paisaje del espacio de estados, estamos en un pico. Los movimientos laterales tampoco ayudan. La ascensión de colinas está **atascada**.

¡Estamos tan cerca de un estado objetivo! Pero un reinicio aleatorio es muy **drástico…** ¿Qué hacemos? 🤔

Búsqueda Tabú

Tabu Search

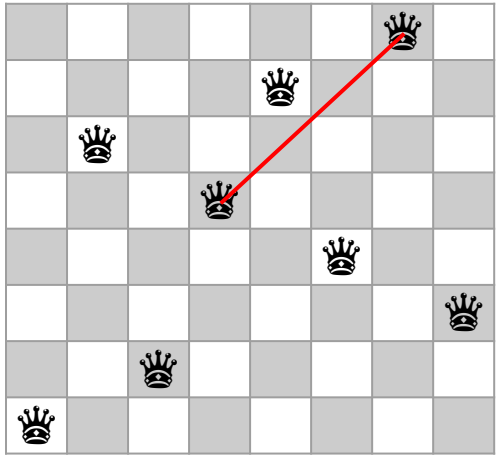
Una **búsqueda tabú** es una **búsqueda local** que incorpora ciertas mejoras al algoritmo de ascensión de colinas para **escapar de máximos locales que no son globales**.

Siempre se mueve al sucesor con mejor valor objetivo, **ya sea mejor, peor o igual que el actual.**

Para mitigar el riesgo de **ciclar**, mantiene una **memoria de corto plazo** que restringe su movimiento, la cual guarda información de las últimas iteraciones. Esta estructura de datos se conoce como **lista tabú.**

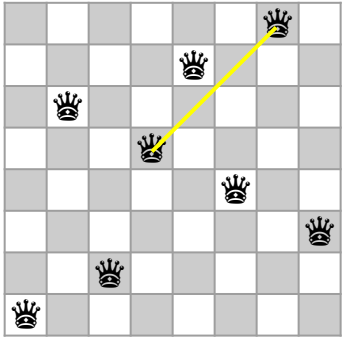
📖 Ejemplo – 8 Reinas

Vamos a aplicar el algoritmo de búsqueda tabú para el problema de las 8-reinas para el siguiente estado inicial:



f(8,3,7,4,2,5,1,6) = 1

📖 Ejemplo – 8 Reinas



f(8,3,7,4,2,5,1,6) = 1

Comenzamos igual que la ascensión de colinas, calculando el valor objetivo de todos los sucesores.

📖 Ejemplo – 8 Reinas

3	3	3	3	2	3	♛	3
3	3	4	2	♛	4	2	4
2	♛	3	3	5	4	2	3
3	2	4	♛	4	4	3	2
3	3	4	3	4	♛	2	3
3	5	3	2	4	3	2	♛
4	3	♛	2	2	3	3	3
♛	3	3	2	2	3	2	3

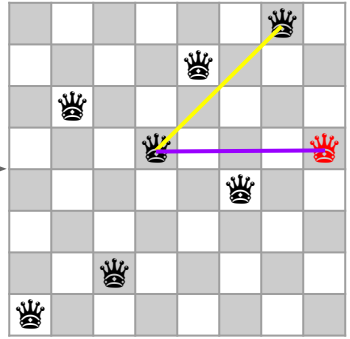
f(8,3,7,4,2,5,1,6) = 1

La búsqueda tabú se mueve siempre al sucesor con mejor valor objetivo, sin importar si es peor al actual. En este caso, debemos elegir cualquiera de los sucesores con valor objetivo igual a 2. Por ejemplo, aplicamos la acción (4,8) que mueve la reina de la columna 8 a la fila 4.

📖 Ejemplo – 8 Reinas

3	3	3	3	2	3	🙁	3
3	3	4	2	🙁	4	2	4
2	🙁	3	3	5	4	2	3
3	2	4	🙁	4	4	3	2
3	3	4	3	4	🙁	2	3
3	5	3	2	4	3	2	🙁
4	3	🙁	2	2	3	3	3
🙁	3	3	2	2	3	2	3

(4,8) →



f(8,3,7,4,2,5,1,6) = 1 f(8,3,7,4,2,5,1,4) = 2

Ahora, ¿a dónde nos movemos? Calculemos el valor objetivo de todos los sucesores.

En este paso, el valor objetivo empeora. **Esto lo diferencia de la ascensión de colinas.**

📖 Ejemplo – 8 Reinas

Lista tabú = {**(6,8)**}

3	3	3	3	2	3	♛	3
3	3	4	2	♛	4	2	4
2	♛	3	3	5	4	2	3
3	2	4	♛	4	4	3	2
3	3	4	3	4	♛	2	3
3	5	3	2	4	3	2	♛
4	3	♛	2	2	3	3	3
♛	3	3	2	2	3	2	3

f(8,3,7,4,2,5,1,6) = 1

(4,8) →

4	4	3	3	4	4	♛	3
4	4	5	**1**	♛	6	3	4
3	♛	4	3	5	5	4	3
5	4	6	♛	6	5	5	♛
4	4	5	3	5	♛	3	3
3	5	3	**1**	4	4	2	**1**
5	4	♛	2	4	4	3	3
♛	4	4	3	3	3	3	3

f(8,3,7,4,2,5,1,4) = 2

Tenemos que elegir alguno de los sucesores con valor objetivo igual a 1. ¿Qué pasaría si aplicamos la acción (6,8) que mueve la reina de la columna 8 a la fila 6? 🤔

En ese caso **repetiremos** el estado anterior.

Para evitarlo, se mantiene en memoria una **lista tabú** con las acciones que no deben aplicarse por **riesgo a ciclar**. Luego de aplicar una acción, se debe agregar a la lista tabú **la acción que la revierte**.

Para este problema, luego de aplicar una acción (i,j) en un estado s = $(f_1,\ldots,f_8)$, se debe agregar a (f_j,j) a la lista tabú.

📖 Ejemplo – 8 Reinas

Lista tabú = {(6,8)}

3	3	3	3	2	3	♛	3
3	3	4	2	♛	4	2	4
2	♛	3	3	5	4	2	3
3	2	4	♛	4	4	3	2
3	3	4	3	4	♛	2	3
3	5	3	2	4	3	2	♛
4	3	♛	2	2	3	3	3
♛	3	3	2	2	3	2	3

f(8,3,7,4,2,5,1,6) = 1

(4,8) →

4	4	3	3	4	4	♛	3
4	4	5	1	♛	6	3	4
3	♛	4	3	5	5	4	3
5	4	6	♛	6	5	5	♛
4	4	5	3	5	♛	3	3
3	5	3	1	4	4	2	-
5	4	♛	2	4	4	3	3
♛	4	4	3	3	3	3	3

f(8,3,7,4,2,5,1,4) = 2

Vamos a marcar con un **-** a aquellos sucesores que se generan con acciones de la lista tabú. Estos sucesores **no pueden elegirse** porque podrían repetir estados.

Elegimos cualquiera de los sucesores con valor objetivo igual a 1. Por ejemplo, aplicamos la acción (6,4).

📖 Ejemplo – 8 Reinas

Lista tabú = {(6,8),**(4,4)**}

3	3	3	3	2	3	♛	3
3	3	4	2	♛	4	2	4
2	♛	3	3	5	4	2	3
3	2	4	♛	4	4	3	2
3	3	4	3	4	♛	2	3
3	5	3	2	4	3	2	♛
4	3	♛	2	2	3	3	3
♛	3	3	2	2	3	2	3

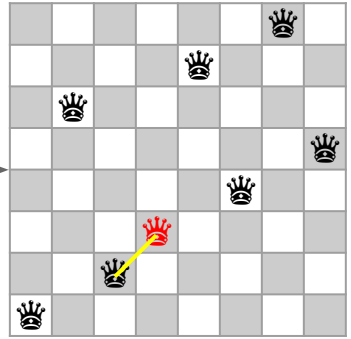
f(8,3,7,4,2,5,1,6) = 1

(4,8) →

4	4	3	3	4	4	♛	3
4	4	5	1	♛	6	3	4
3	♛	4	3	5	5	4	3
5	4	6	♛	6	5	5	♛
4	4	5	3	5	♛	3	3
3	5	3	**1**	4	4	2	-
5	4	♛	2	4	4	3	3
♛	4	4	3	3	3	3	3

f(8,3,7,4,2,5,1,4) = 2

(6,4) →



f(8,3,7,6,2,5,1,4) = 1

📖 Ejemplo – 8 Reinas

Lista tabú = {(6,8),(4,4)}

3	3	3	3	2	3	♛	3
3	3	4	2	♛	4	2	4
2	♛	3	3	5	4	2	3
3	2	4	♛	4	4	3	2
3	3	4	3	4	♛	2	3
3	5	3	2	4	3	2	♛
4	3	♛	2	2	3	3	3
♛	3	3	2	2	3	2	3

f(8,3,7,4,2,5,1,6) = 1

(4,8) →

4	4	3	3	4	4	♛	3
4	4	5	1	♛	6	3	4
3	♛	4	3	5	5	4	3
5	4	6	♛	6	5	5	♛
4	4	5	3	5	♛	3	3
3	5	3	1	4	4	2	-
5	4	♛	2	4	4	3	3
♛	4	4	3	3	3	3	3

f(8,3,7,4,2,5,1,4) = 2

(6,4) →

2	3	1	3	3	3	♛	3
3	2	3	1	♛	4	3	5
3	♛	1	3	3	4	5	3
3	3	3	-	4	4	4	♛
3	3	3	3	4	♛	3	3
3	4	2	♛	4	3	3	-
3	3	♛	2	4	3	2	3
♛	4	2	3	2	3	3	2

f(8,3,7,6,2,5,1,4) = 1

Elegimos cualquiera de los sucesores con valor objetivo igual a 1. Es decir, hacemos un **movimiento lateral**. Por ejemplo, aplicamos la acción (3,3).

📖 Ejemplo – 8 Reinas

Lista tabú = {(6,8),(4,4),**(7,3)**}

3	3	3	3	2	3	🩸	3
3	3	4	2	🩸	4	2	4
2	🩸	3	3	5	4	2	3
3	2	4	🩸	4	4	3	2
3	3	4	3	4	🩸	2	3
3	5	3	2	4	3	2	🩸
4	3	🩸	2	2	3	3	3
🩸	3	3	2	2	3	2	3

f(8,3,7,4,2,5,1,6) = 1

(4,8) →

4	4	3	3	4	4	🩸	3
4	4	5	1	🩸	6	3	4
3	🩸	4	3	5	5	4	3
5	4	6	🩸	6	5	5	🩸
4	4	5	3	5	🩸	3	3
3	5	3	1	4	4	2	-
5	4	🩸	2	4	4	3	3
🩸	4	4	3	3	3	3	3

f(8,3,7,4,2,5,1,4) = 2

(6,4) →

2	3	1	3	3	3	🩸	3
3	2	3	1	🩸	4	3	5
3	🩸	**1**	3	3	4	5	3
3	3	3	-	4	4	4	🩸
3	3	3	3	4	🩸	3	3
3	4	2	🩸	4	3	3	-
3	3	🩸	2	4	3	2	3
🩸	4	2	3	2	3	3	2

f(8,3,7,6,2,5,1,4) = 1

(3,3) →

						🩸	
				🩸			
	🩸	🩸					
							🩸
					🩸		
			🩸				
🩸							

f(8,3,3,6,2,5,1,4) = 1

📖 Ejemplo – 8 Reinas

Lista tabú = {(6,8),(4,4),(7,3)}

3	3	3	3	2	3	🔴	3
3	3	4	2	🔴	4	2	4
2	🔴	3	3	5	4	2	3
3	2	4	🔴	4	4	3	2
3	3	4	3	4	🔴	2	3
3	5	3	2	4	3	2	🔴
4	3	🔴	2	2	3	3	3
🔴	3	3	2	2	3	2	3

f(8,3,7,4,2,5,1,6) = 1

(4,8) →

4	4	3	3	4	4	🔴	3
4	4	5	1	🔴	6	3	4
3	🔴	4	3	5	5	4	3
5	4	6	🔴	6	5	5	🔴
4	4	5	3	5	🔴	3	3
3	5	3	1	4	4	2	-
5	4	🔴	2	4	4	3	3
🔴	4	4	3	3	3	3	3

f(8,3,7,4,2,5,1,4) = 2

(6,4) →

2	3	1	3	3	3	🔴	3
3	2	3	1	🔴	4	3	5
3	🔴	1	3	3	4	5	3
3	3	3	-	4	4	4	🔴
3	3	3	3	4	🔴	3	3
3	4	2	🔴	4	3	3	-
3	3	🔴	2	4	3	2	3
🔴	4	2	3	2	3	3	2

f(8,3,7,6,2,5,1,4) = 1

(3,3) →

3	2	1	4	4	3	🔴	3
3	2	3	3	🔴	4	3	4
4	🔴	🔴	5	4	5	5	4
3	3	3	-	4	3	4	🔴
3	2	3	4	4	🔴	3	3
3	2	2	🔴	4	4	3	-
2	1	-	2	3	2	2	2
🔴	2	2	3	2	3	3	3

f(8,3,3,6,2,5,1,4) = 1

Elegimos cualquiera de los sucesores con valor objetivo igual a 1. Es decir, hacemos otro **movimiento lateral**. Por ejemplo, aplicamos la acción (7,2).

📖 Ejemplo – 8 Reinas

Lista tabú = {(6,8),(4,4),(7,3),**(3,2)**}

3	3	3	3	2	3	♛	3
3	3	4	2	♛	4	2	4
2	♛	3	3	5	4	2	3
3	2	4	♛	4	4	3	2
3	3	4	3	4	♛	2	3
3	5	3	2	4	3	2	♛
4	3	♛	2	2	3	3	3
♛	3	3	2	2	3	2	3

f(8,3,7,4,2,5,1,6) = 1

(4,8) →

4	4	3	3	4	4	♛	3
4	4	5	1	♛	6	3	4
3	♛	4	3	5	5	4	3
5	4	6	♛	6	5	5	♛
4	4	5	3	5	♛	3	3
3	5	3	1	4	4	2	-
5	4	♛	2	4	4	3	3
♛	4	4	3	3	3	3	3

f(8,3,7,4,2,5,1,4) = 2

(6,4) →

2	3	1	3	3	3	♛	3
3	2	3	1	♛	4	3	5
3	♛	1	3	3	4	5	3
3	3	3	-	4	4	4	♛
3	3	3	3	4	♛	3	3
3	4	2	♛	4	3	3	-
3	3	♛	2	4	3	2	3
♛	4	2	3	2	3	3	2

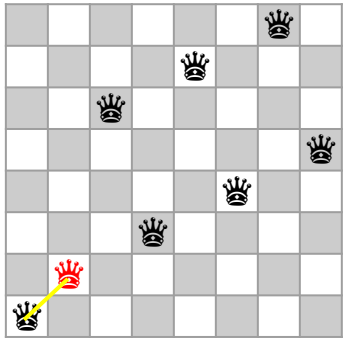
f(8,3,7,6,2,5,1,4) = 1

(3,3) →

3	2	1	4	4	3	♛	3
3	2	3	3	♛	4	3	4
4	♛	♛	5	4	5	5	4
3	3	3	-	4	3	4	♛
3	2	3	4	4	♛	3	3
3	2	2	♛	4	4	3	-
2	1	-	2	3	2	2	2
♛	2	2	3	2	3	3	3

f(8,3,3,6,2,5,1,4) = 1

(7,2) →



f(8,7,3,6,2,5,1,4) = 1

📖 Ejemplo – 8 Reinas

Lista tabú = {(6,8),(4,4),(7,3),(3,2)}

3	3	3	3	2	3	♛	3
3	3	4	2	♛	4	2	4
2	♛	3	3	5	4	2	3
3	2	4	♛	4	4	3	2
3	3	4	3	4	♛	2	3
3	5	3	2	4	3	2	♛
4	3	♛	2	2	3	3	3
♛	3	3	2	2	3	2	3

f(8,3,7,4,2,5,1,6) = 1

(4,8) →

4	4	3	3	4	4	♛	3
4	4	5	1	♛	6	3	4
3	♛	4	3	5	5	4	3
5	4	6	♛	6	5	5	♛
4	4	5	3	5	♛	3	3
3	5	3	1	4	4	2	-
5	4	♛	2	4	4	3	3
♛	4	4	3	3	3	3	3

f(8,3,7,4,2,5,1,4) = 2

(6,4) →

2	3	1	3	3	3	♛	3
3	2	3	1	♛	4	3	5
3	♛	1	3	3	4	5	3
3	3	3	-	4	4	4	♛
3	3	3	3	4	♛	3	3
3	4	2	♛	4	3	3	-
3	3	♛	2	4	3	2	3
♛	4	2	3	2	3	3	2

f(8,3,7,6,2,5,1,4) = 1

(3,3) →

3	2	1	4	4	3	♛	3
3	2	3	3	♛	4	3	4
4	♛	♛	5	4	5	5	4
3	3	3	-	4	3	4	♛
3	2	3	4	4	♛	3	3
3	2	2	♛	4	4	3	-
2	1	-	2	3	2	2	2
♛	2	2	3	2	3	3	3

f(8,3,3,6,2,5,1,4) = 1

(7,2) ↙

2	2	2	3	4	3	♛	4
1	2	3	3	♛	4	4	4
2	-	♛	4	3	5	4	3
1	3	3	-	5	3	4	♛
2	2	4	4	4	♛	3	3
3	2	4	♛	3	4	3	-
2	♛	-	3	4	2	3	3
♛	2	4	3	2	3	2	3

f(8,7,3,6,2,5,1,4) = 1

Elegimos cualquiera de los sucesores con valor objetivo igual a 1. Es decir, hacemos otro **movimiento lateral**. Por ejemplo, aplicamos la acción (2,1).

📖 Ejemplo – 8 Reinas

Lista tabú = {(6,8),(4,4),(7,3),(3,2),**(8,1)**}

3	3	3	3	2	3	♛	3
3	3	4	2	♛	4	2	4
2	♛	3	3	5	4	2	3
3	2	4	♛	4	4	3	2
3	3	4	3	4	♛	2	3
3	5	3	2	4	3	2	♛
4	3	♛	2	2	3	3	3
♛	3	3	2	2	3	2	3

f(8,3,7,4,2,5,1,6) = 1

(4,8)

4	4	3	3	4	4	♛	3
4	4	5	1	♛	6	3	4
3	♛	4	3	5	5	4	3
5	4	6	♛	6	5	5	♛
4	4	5	3	5	♛	3	3
3	5	3	1	4	4	2	-
5	4	♛	2	4	4	3	3
♛	4	4	3	3	3	3	3

f(8,3,7,4,2,5,1,4) = 2

(6,4)

2	3	1	3	3	3	♛	3
3	2	3	1	♛	4	3	5
3	♛	1	3	3	4	5	3
3	3	3	-	4	4	4	♛
3	3	3	3	4	♛	3	3
3	4	2	♛	4	3	3	-
3	3	♛	2	4	3	2	3
♛	4	2	3	2	3	3	2

f(8,3,7,6,2,5,1,4) = 1

(3,3)

3	2	1	4	4	3	♛	3
3	2	3	3	♛	4	3	4
4	♛	♛	5	4	5	5	4
3	3	3	-	4	3	4	♛
3	2	3	4	4	♛	3	3
3	2	2	♛	4	4	3	-
2	1	-	2	3	2	2	2
♛	2	2	3	2	3	3	3

f(8,3,3,6,2,5,1,4) = 1

(7,2)

2	2	2	3	4	3	♛	4
1	2	3	3	♛	4	4	4
2	-	♛	4	3	5	4	3
1	3	3	-	5	3	4	♛
2	2	4	4	4	♛	3	3
3	2	4	♛	3	4	3	-
2	♛	-	3	4	2	3	3
♛	2	4	3	2	3	2	3

f(8,7,3,6,2,5,1,4) = 1

(2,1)

						♛	
♛				♛			
		♛					
							♛
					♛		
			♛				
	♛						

f(2,7,3,6,2,5,1,4) = 1

📖 Ejemplo – 8 Reinas

Lista tabú = {(6,8),(4,4),(7,3),(3,2),(8,1)}

3	3	3	3	2	3	♛	3
3	3	4	2	♛	4	2	4
2	♛	3	3	5	4	2	3
3	2	4	♛	4	4	3	2
3	3	4	3	4	♛	2	3
3	5	3	2	4	3	2	♛
4	3	♛	2	2	3	3	3
♛	3	3	2	2	3	2	3

f(8,3,7,4,2,5,1,6) = 1

(4,8) →

4	4	3	3	4	4	♛	3
4	4	5	1	♛	6	3	4
3	♛	4	3	5	5	4	3
5	4	6	♛	6	5	5	♛
4	4	5	3	5	♛	3	3
3	5	3	1	4	4	2	-
5	4	♛	2	4	4	3	3
♛	4	4	3	3	3	3	3

f(8,3,7,4,2,5,1,4) = 2

(6,4) →

2	3	1	3	3	3	♛	3
3	2	3	1	♛	4	3	5
3	♛	1	3	3	4	5	3
3	3	3	-	4	4	4	♛
3	3	3	3	4	♛	3	3
3	4	2	♛	4	3	3	-
3	3	♛	2	4	3	2	3
♛	4	2	3	2	3	3	2

f(8,3,7,6,2,5,1,4) = 1

(3,3) →

3	2	1	4	4	3	♛	3
3	2	3	3	♛	4	3	4
4	♛	♛	5	4	5	5	4
3	3	3	-	4	3	4	♛
3	2	3	4	4	♛	3	3
3	2	2	♛	4	4	3	-
2	1	-	2	3	2	2	2
♛	2	2	3	2	3	3	3

f(8,3,3,6,2,5,1,4) = 1

(7,2) ↙

2	2	2	3	4	3	♛	4
1	2	3	3	♛	4	4	4
2	-	♛	4	3	5	4	3
1	3	3	-	5	3	4	♛
2	2	4	4	4	♛	3	3
3	2	4	♛	3	4	3	-
2	♛	-	3	4	2	3	3
♛	2	4	3	2	3	2	3

f(8,7,3,6,2,5,1,4) = 1

(2,1) ←

2	4	2	3	3	3	♛	3
♛	4	4	4	♛	5	4	5
2	-	♛	4	2	4	4	3
1	4	4	-	3	3	4	♛
2	3	4	4	3	♛	3	3
3	3	3	♛	3	4	3	-
2	♛	-	3	3	3	3	3
-	2	3	2	0	2	2	2

f(2,7,3,6,2,5,1,4) = 1

Elegimos el sucesor con valor objetivo igual a 0.

📖 Ejemplo – 8 Reinas

Lista tabú = {(6,8),(4,4),(7,3),(3,2),(8,1),(2,5)}

3	3	3	3	2	3	♛	3
3	3	4	2	♛	4	2	4
2	♛	3	3	5	4	2	3
3	2	4	♛	4	4	3	2
3	3	4	3	4	♛	2	3
3	5	3	2	[illegible]	[illegible]	[illegible]	[illegible]
4	3	♛	2	[illegible]	[illegible]	[illegible]	[illegible]
♛	3	3	2	[illegible]	[illegible]	[illegible]	[illegible]

f(8,3,7,4,2,5,1,[illegible]) = 1

(4,8) →

4	4	3	3	4	4	♛	3
4	4	5	1	♛	6	3	4
3	♛	4	3	5	5	4	3
5	4	6	♛	6	5	5	♛
4	4	5	3	5	♛	3	3
[illegible]	[illegible]	[illegible]	[illegible]	4	4	2	-
[illegible]	[illegible]	[illegible]	[illegible]	4	4	3	3
[illegible]	[illegible]	[illegible]	[illegible]	3	3	3	3

f(8,3,7,4,2,5,1,4) = 2

(6,4) →

2	3	1	3	3	3	♛	3
3	2	3	1	♛	4	3	5
3	♛	1	3	3	4	5	3
3	3	3	-	4	4	4	♛
3	3	3	3	4	♛	3	3
3	4	2	♛	4	3	3	-
3	3	♛	2	4	3	2	3
♛	4	2	3	2	3	3	2

f(8,3,7,6,2,5,1,4) = 1

(3,3) →

3	2	1	4	4	3	♛	3
3	2	3	3	♛	4	3	4
4	♛	♛	5	4	5	5	4
3	3	3	-	4	3	4	♛
3	2	3	4	4	♛	3	3
3	2	2	♛	4	4	3	-
2	1	-	2	3	2	2	2
♛	2	2	3	2	3	3	3

f(8,3,3,6,2,5,1,4) = 1

(7,2) ↙

2	2	2	3	4	3	♛	4
1	2	3	3	♛	4	4	4
2	-	♛	4	3	5	4	3
1	3	3	-	5	3	4	♛
2	2	4	4	4	♛	3	3
3	2	4	♛	3	4	3	-
2	♛	-	3	4	2	3	3
♛	2	4	3	2	3	2	3

f(8,7,3,6,2,5,1,4) = 1

(2,1) ←

2	4	2	3	3	3	♛	3
♛	4	4	4	♛	5	4	5
2	-	♛	4	2	4	4	3
1	4	4	-	3	3	4	♛
2	3	4	4	3	♛	3	3
3	3	3	♛	3	4	3	-
2	♛	-	3	3	3	3	3
-	2	3	2	0	2	2	2

f(2,7,3,6,2,5,1,4) = 1

(8,5) ←

						♛	
♛							
		♛					
							♛
					♛		
			♛				
	♛						
				♛			

f(2,7,3,6,8,5,1,4) = 0

¡Encontramos un estado objetivo!

💻 Algoritmo de búsqueda tabú

```
function BÚSQUEDA-TABÚ(problema) return estado
   actual ← problema.estado-inicial
   mejor ← actual
   tabu ← inicialmente vacía
   while no se cumpla el criterio de parada do
        accion ←  MAX-ACCION(problema, actual, tabu)
        sucesor ← problema.resultado(actual, accion)
        if problema.f(mejor) < problema.f(sucesor) then mejor ← sucesor
        actualizar la lista tabú
        actual ← sucesor
   return mejor
```

Ahora, MAX-ACCION también toma la lista tabú. La idea es que retorne la mejor acción entre aquellas que no son tabú.

💻 Algoritmo de búsqueda tabú

```
function BÚSQUEDA-TABÚ(problema) return estado
    actual ← problema.estado-inicial
    mejor ← actual
    tabu ← inicialmente vacía
    while no se cumpla el criterio de parada do
        accion ←  MAX-ACCION(problema, actual, tabu)
        sucesor ← problema.resultado(actual, accion)
        if problema.f(mejor) < problema.f(sucesor) then mejor ← sucesor
        actualizar la lista tabú
        actual ← sucesor
    return mejor
```

Observación 1. Como la búsqueda puede moverse hacia estados con peor valor objetivo, se mantiene en memoria el estado con **mejor** valor objetivo hasta el momento. Notar que puede diferir del estado actual.

Observación 2. Se actualiza el estado actual por el sucesor, **haya o no** mejorado el valor objetivo.

Observación 3. Es necesario un criterio de parada, dado que la búsqueda podría no terminar.

¿Cómo construir la lista tabú y qué criterio de parada elegir? 🤔

Lista tabú

¿Qué elementos se suelen almacenar en la lista tabú?

1. **Acciones**.
2. **Estados**.
3. **Propiedades**.

A continuación veremos con más detalle cada una de estas alternativas.

1. Acciones

Acciones que podrían revertir acciones realizadas recientemente.

📖 **Ejemplo - 8 Reinas.**

Suponer que inicialmente la reina de la columna j está en la fila f_j y se la mueve a la fila i (con la acción (i,j)). Tenemos dos opciones:

1. Como ya vimos, una posibilidad es agregar a la lista tabú la acción (f_j,j).
2. Otra posibilidad **más restrictiva** es prohibir mover esa reina por completo. Es decir, agregar a la lista tabú las acciones (1,j), …, (8,j).

2. Estados

Estados que hayan sido visitados recientemente.

En general, **restringe menos** el espacio de búsqueda pero a costa de un mayor consumo de **tiempo** y **memoria**.

Los estados suelen ser más pesados que las acciones y comparar dos estados por igualdad suele consumir más tiempo.

📖 **Ejemplo - 8 Reinas.**

Un estado es una tupla de 8 números $(f_1,\ldots,f_8)$, mientras que una acción es un par de números (i,j).

3. Propiedades

Propiedades sobre atributos de los estados o de las acciones.

Los estados sucesores que verifiquen alguna de las propiedades de la lista tabú son ignorados.

📖 **Ejemplo - 8 Reinas.**

Las reinas de las columnas j_1 y j_2 se atacan mutuamente.

Lista tabú

Ahora que ya sabemos qué almacenar en la lista tabú, tenemos que decidir por cuánto tiempo los almacenamos…

Pregunta. ¿Qué pasaría si los elementos de la lista tabú se mantienen allí para siempre? 🤔

Lista tabú

Ahora que ya sabemos qué almacenar en la lista tabú, tenemos que decidir por cuánto tiempo los almacenamos…

Pregunta. ¿Qué pasaría si los elementos de la lista tabú se mantienen allí para siempre? 🤔

Respuesta.

1. Consumo de memoria **excesivo**. Dejaría de ser búsqueda local.
2. Mayor probabilidad de **atascarse**. Podría no existir un sucesor que no sea tabú.

Lista tabú

La lista tabú debe funcionar como una **memoria de corto plazo**, almacenando sólo la información reciente y no toda la historia del algoritmo.

Existen dos enfoques.

1. **Capacidad limitada.**

   Si al agregar un nuevo elemento a la lista se excede su capacidad, entonces se borra de la lista el elemento más viejo. Es decir, la lista tabú funciona como una **cola**.

2. **Tenor de tabú.**

   Luego de agregar un nuevo elemento a la lista, se filtra la lista y se borran aquellos que hayan estado por más iteraciones del límite establecido. A este valor se lo conoce como **tenor de tabú**.

Criterio de parada

---

1. Tras cierto número de **iteraciones totales** (o tiempo de CPU).
2. Tras cierto número de **iteraciones sin mejoras** en el **mejor** valor objetivo.
3. Cuando la función objetivo sobrepasa cierto **valor umbral**.

Componentes adicionales

En problemas difíciles se suelen incorporar a la búsqueda tabú algunas **componentes adicionales** para mejorar su efectividad.

Entre las más comunes se encuentran:

1. **Criterio de aspiración**.
2. **Diversificación**.

1. Criterio de aspiración

La lista tabú puede volverse demasiado restrictiva:

- Puede prohibir acciones atractivas, incluso cuando no hay riesgo de ciclar, o
- Puede conducir a un estancamiento en la búsqueda.

Un **criterio de aspiración** permite ignorar tabúes.

El más sencillo consiste en permitir una acción, incluso si es tabú, siempre que el valor objetivo del estado sucesor sea mayor al del mejor estado encontrado hasta el momento (notar que no hay riesgo de ciclar, ¿por qué?).

2. Diversificación

Los algoritmos de búsqueda local, en particular la búsqueda tabú, tienden a pasar la mayor parte del tiempo en una porción restringida del espacio de estados, ignorando quizás regiones más interesantes.

La **diversificación** es un mecanismo que fuerza la búsqueda en regiones inexploradas. Típicamente usa **memoria de largo plazo**.

Lo más sencillo es la **memoria de frecuencia**, que registra por cuántas iteraciones se preservaron algunos componentes de la solución.

📖 **Ejemplo - 8 Reinas.**

Memoria de frecuencia. Para toda fila i y columna j, se almacena en una tabla el número de iteraciones en donde la reina de la columna j estuvo en la fila i.

2. Diversificación

A partir de esta información, existen dos estrategias de diversificación.

1. **De reinicio**. Reiniciar la búsqueda desde un estado con componentes poco frecuentes.
2. **Continua**. A la hora de elegir un estado sucesor, priorizar aquellos con componentes poco frecuentes (agregar un término de frecuencia a la función objetivo).

📖 **Ejemplo - 8 Reinas.**

Diversificación de reinicio. Tras cierto número de iteraciones, reiniciar la búsqueda tabú desde el estado que tiene a cada reina en su fila menos frecuente.

Seteo de parámetros

Pregunta. A la hora de implementar una búsqueda tabú…

- ¿Qué almacenar en la lista tabú: estados, acciones o propiedades?
- ¿Por cuánto tiempo almacenarlos: capacidad o tenor de tabú de la lista?
- ¿Cuándo parar: número de iteraciones, o número de iteraciones sin mejora, o tiempo límite, etc?
- …

Seteo de parámetros

Pregunta. A la hora de implementar una búsqueda tabú…

- ¿Qué almacenar en la lista tabú: estados, acciones o propiedades?
- ¿Por cuánto tiempo almacenarlos: capacidad o tenor de tabú de la lista?
- ¿Cuándo parar: número de iteraciones, o número de iteraciones sin mejora, o tiempo límite, etc?
- …

Respuesta. No es obvio, **se determina experimentalmente**.

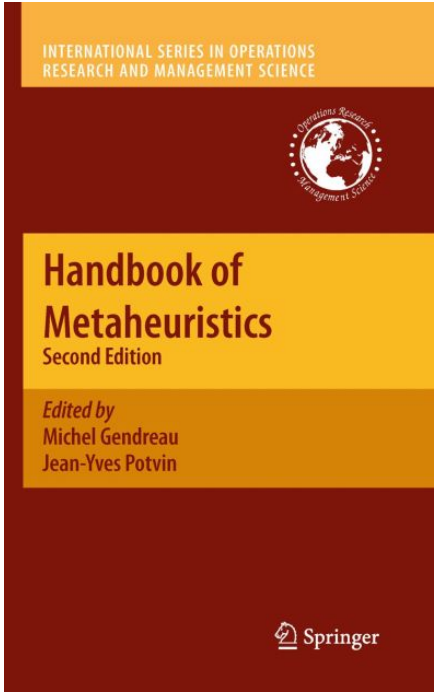
Los algoritmos de búsqueda tabú suelen ser extremadamente sensibles al seteo de los parámetros.

No es extraño que la performance mejore/empeore drásticamente al cambiar el valor de algunos parámetros.

Performance de BÚSQUEDA-TABÚ

- Bajo consumo de **memoria**, aunque mayor que ASCENSIÓN-COLINAS.
- No se **atasca** en máximos locales porque puede moverse a un sucesor con peor valor objetivo.
- Se mantiene una **lista tabú** para reducir el riesgo de **ciclar**, aunque no lo elimina por completo.
- La **terminación** se garantiza con un **criterio de parada**.
- En general, no hay garantía de que el estado retornado sea objetivo (**completitud**), ni que el máximo alcanzado sea global (**optimalidad**).

Bibliografía



Capítulo 2. Tabu Search

Michel Gendreau and Jean-Yves Potvin

Búsqueda local en espacios continuos

En el mundo real, las magnitudes (longitud, masa, tiempo, etc.) asumen valores continuos.

Ninguno de los algoritmos de búsqueda local vistos hasta ahora son aplicables a problemas con espacios de estados continuos.

En estos casos, los estados tienen **infinitos sucesores**.

¿Cómo buscamos soluciones en espacios continuos?

Problema de ubicación de aeropuerto

Objetivo. Ubicar un aeropuerto, de forma tal que la suma de las distancias al cuadrado de cada ciudad del mapa al aeropuerto sea mínima.

Formulación

En esta formulación, un **estado** es un punto (x,y) de R^2 con la ubicación del aeropuerto, donde *x* e *y* son números reales.

La número total de estados es **infinito**.

Una acción es un vector (i,j) de R^2 que representa el desplazamiento del aeropuerto.

El número de sucesores de un estado también es **infinito**.

Formulación

- **Estado inicial**. Cualquier estado (x,y).
- **Acciones**. Dado un estado (x,y),

  ACCIONES(x,y) = { (i,j): i,j ∈ R\{(0,0)} }.
- **Modelo de transiciones**. Dado un estado (x,y) y una acción (i,j),

  RESULTADO((x,y),(i,j)) = $(x+i,y+j)$.
- **Test objetivo.** Todo estado es objetivo.
- **Función objetivo**.

$$f(x,y) = -\sum_{c \in C} \left((x - x_c)^2 + (y - y_c)^2\right)$$

Lo pensamos como un problema de maximización.

Donde C es el conjunto de ciudades y las constantes (x_c,y_c) son las coordenadas de la ciudad $c \in C$.

📖 Ejemplo – Aeropuerto

Suponiendo que las ciudades tienen las siguientes coordenadas.

La función objetivo se simplifica a:

$$f(x, y) = -6x^2 + 12x - 6y^2 + 6y - 13$$

Discretizar vs. no discretizar

Veremos dos técnicas para resolver problemas de este tipo.

1. **Discretizar** el espacio de estados.
2. **No discretizar** el espacio de estados.

1. Discretizar

Discretizar consiste en restringir el problema de forma tal que, para todo estado S, el conjunto

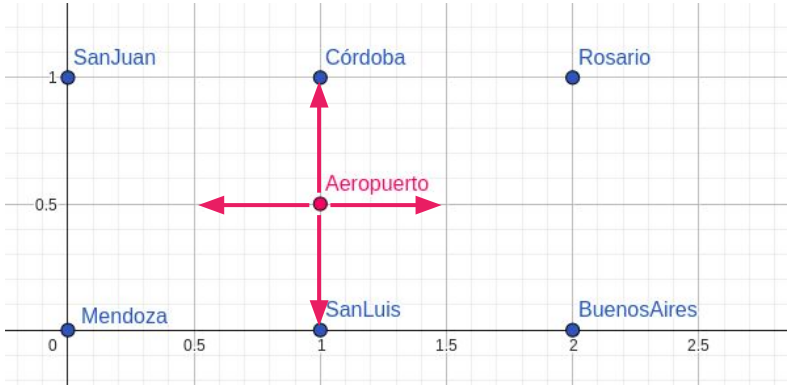
$$\text{ACCIONES}(S)$$

sea **finito**.

Luego, podemos resolverlo con cualquiera de los algoritmos de búsqueda local descritos anteriormente.

📖 Ejemplo – Aeropuerto

El aeropuerto se puede mover únicamente vertical u horizontalmente en una cantidad fija $\pm\delta$.



$\delta=0.5$

Es decir,

$$\text{ACCIONES}(x,y) = \{(\delta,0),(-\delta,0),(0,\delta),(0,-\delta)\}.$$

2. No discretizar

Buscar máximos locales en espacios de estados continuos requiere repasar algunos conceptos de **cálculo**.

La literatura sobre el tema es enorme, veremos una muy breve introducción.

Gradiente

- Sea un campo escalar
$$f : \mathbb{R}^n \to \mathbb{R}.$$

- El **gradiente** de *f*, notado ∇(f), es el campo vectorial

$$\nabla(f)(x_1, \ldots, x_n) = \left(\frac{\partial f}{\partial x_1}(x_1, \ldots, x_n), \ldots, \frac{\partial f}{\partial x_n}(x_1, \ldots, x_n) \right).$$

donde $\frac{\partial f}{\partial x_i}$ es la **derivada parcial** de *f* respecto a x_i .

📖 Ejemplo – Aeropuerto

El gradiente de la función objetivo:

$$f(x, y) = -6x^2 + 12x - 6y^2 + 6y - 13$$

es el campo vectorial:

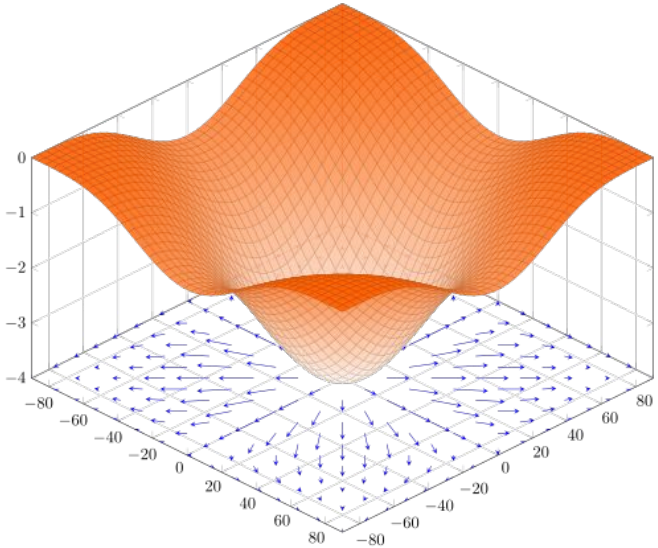
$$\bigtriangledown f(x, y) = (-12x + 12, -12y + 6)$$

Interpretación geométrica

Dado un punto $(a_1, \ldots, a_n) \in \mathbb{R}^n$, el vector $\bigtriangledown(f)(a_1, \ldots, a_n)$ muestra la dirección en la que hay que moverse estando en el punto $(a_1, \ldots, a_n)$ para aumentar el valor de *f* lo más rápido posible y su módulo indica cuán rápido aumenta.

Ejemplo.

$f(x) = -(\cos^2(x) + \cos^2(y))^2$

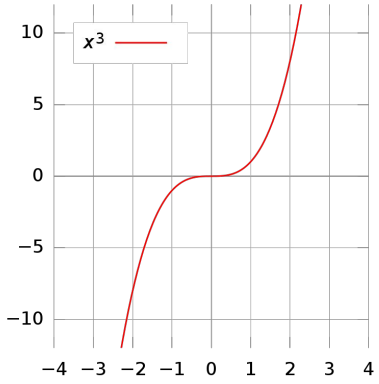


Puntos estacionarios

- Dada una función diferenciable (derivable en cualquier dirección), un **punto estacionario** es un punto del dominio donde todas las derivadas parciales evalúan a 0. Es decir,

$$\bigtriangledown f(x_1, \ldots, x_n) = (0, \ldots, 0)$$

- Un punto estacionario puede ser:
    - un **máximo local**,
    - un **mínimo local**,
    - o un **punto de silla**.
- Algunas veces, es posible calcularlos analíticamente.



📖 Ejemplo – Aeropuerto

Los puntos estacionarios de la función objetivo

$$f(x, y) = -6x^2 + 12x - 6y^2 + 6y - 13$$

se obtienen resolviendo el sistema de ecuaciones

$$\bigtriangledown f(x, y) = (0, 0)$$

es decir,

$$\begin{cases} -12x + 12 = 0 \\ -12y + 6 = 0 \end{cases}$$

📖 Ejemplo – Aeropuerto

La única solución del sistema

$$\begin{cases} -12x + 12 = 0 \\ -12y + 6 = 0 \end{cases}$$

es el punto p = (1,0.5).

Claramente, p es un máximo global de f porque es el centro geométrico del rectángulo cuyos vértices están en Mendoza, San Juan, Rosario y Buenos Aires.

Existen criterios para clasificar un punto estacionario. Por ejemplo, el criterio de la **derivada segunda**.

Aproximación numérica

- Desafortunadamente, no siempre es posible hallar los puntos estacionarios de forma analítica.
  - Por ejemplo, la ecuación $\bigtriangledown f(x_1, \ldots, x_n) = (0, \ldots, 0)$ podría ser difícil de despejar, tener muchas variables, infinitas soluciones, etc.
- Los máximos y mínimos locales se pueden aproximar **numéricamente**.

Ascenso de gradiente (o ascenso por el gradiente)

El algoritmo de **ascensión de gradiente**:

1. Parte de un punto x del dominio.
2. En cada iteración se mueve al punto

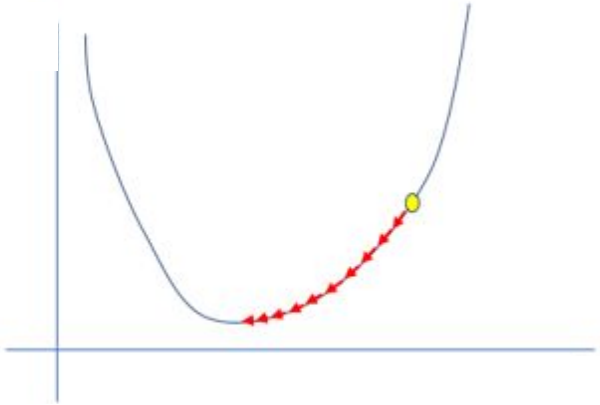
$$x \leftarrow x + \alpha . \nabla f(x),$$

   donde α es una constante pequeña, llamada **tamaño de paso**.

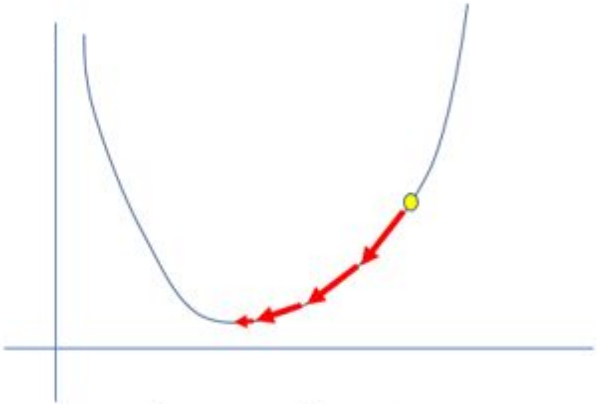
Si se busca un mínimo local (descenso de gradiente)

$$x \leftarrow x - \alpha . \nabla f(x).$$

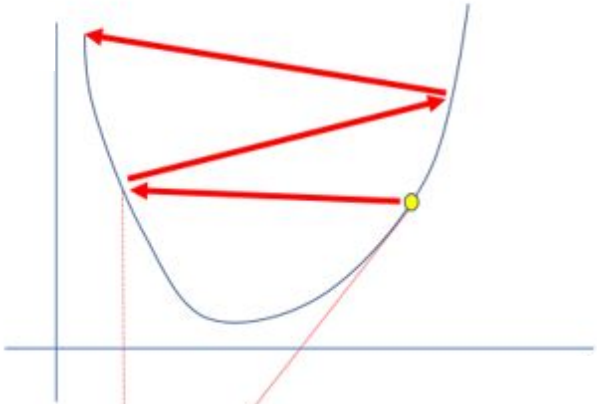
📖 Ejemplos



α muy chico,
requiere muchas
iteraciones



α adecuado,
requiere menos
iteraciones.



α muy grande,
puede escapar del
mínimo local.

Funciones no diferenciables

- Las funciones no diferenciables son más difíciles de tratar.
- No es posible obtener una fórmula cerrada para ∇f.
- Si el gradiente se puede calcular **localmente**, entonces todavía es posible aplicar la ascensión de gradiente.

📖 Ejemplo – Ubicar dos aeropuertos

El objetivo es ubicar 2 aeropuertos de forma tal que la suma de las distancias al cuadrado de cada ciudad del mapa al aeropuerto más cercano sea mínima.

📖 Ejemplo – Colocar dos aeropuertos

La función objetivo ahora es:

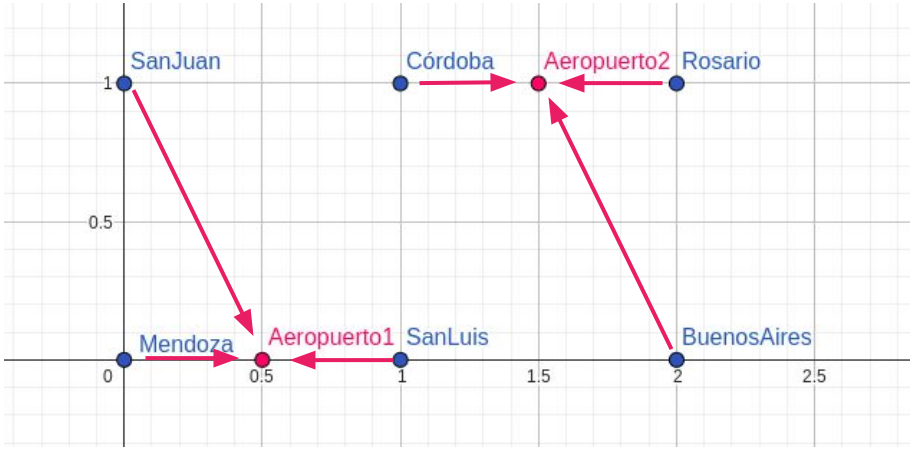
$$f(x_1, y_1, x_2, y_2) = -\sum_{c \in C} \min((x_1 - x_c)^2 + (y_1 - y_c)^2, (x_2 - x_c)^2 + (y_2 - y_c)^2)$$

donde (x_1,y_1) es la coordenada del primer aeropuerto y (x_2,y_2) es la coordenada del segundo aeropuerto.

Esta función no es diferenciable **globalmente**, pero sí lo es **localmente** para una asignación fija de las ciudades a su aeropuerto más cercano.

📖 Ejemplo – Colocar dos aeropuertos

Suponer que las ciudades se ubican en las siguientes coordenadas.



Si la coordenada del primer aeropuerto es (0.5,0) y la del segundo aeropuerto es (1.5,1), entonces es fácil determinar cuál es el aeropuerto más cercano para cada ciudad. Las flechas que salen de cada ciudad apuntan al aeropuerto más cercano.

📖 Ejemplo – Colocar dos aeropuertos

- La función objetivo en un punto responde a la ley:

$$f(x_1, y_1, x_2, y_v) = -\sum_{i=1}^{i=2} \sum_{c \in C_i} \left((x_i - x_c)^2 + (y_i - y_c)^2 \right),$$

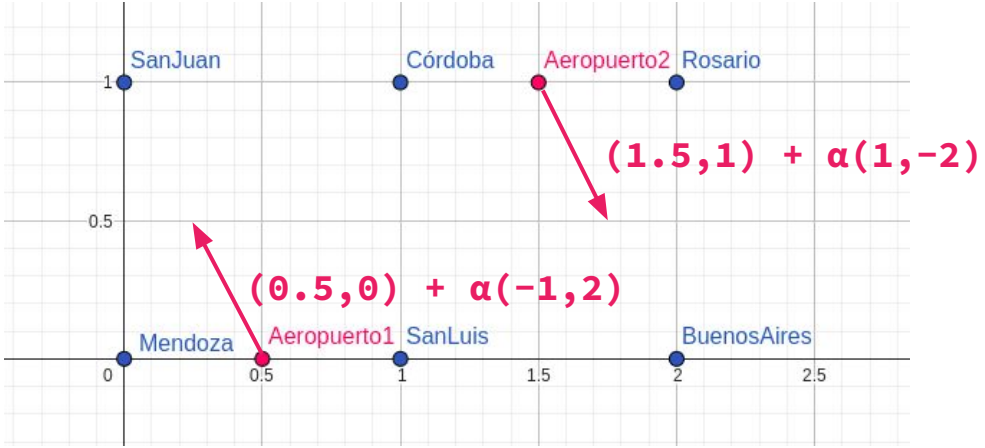
  donde C_i es el conjunto de ciudades que tienen al aeropuerto de la coordenada (x_i, y_i) como más cercano.

- El gradiente en un punto responde a la ley:

$$\nabla(f)(x_1, y_1, x_2, y_v) = \left(-2 \sum_{c \in C_1} (x_1 - x_c), -2 \sum_{c \in C_1} (y_1 - y_c), -2 \sum_{c \in C_2} (x_2 - x_c), -2 \sum_{c \in C_2} (y_2 - y_c) \right).$$

📖 Ejemplo – Ubicar dos aeropuertos

En el punto (0.5,0,1.5,1), el gradiente es (-1,2,1,-2).



📚 Resumen

- La búsqueda tabú recorre el paisaje del espacio de estados de forma más intrincada que la ascensión de colinas.
- Incorpora un mecanismo para escapar de máximos locales y una memoria de corto plazo para evitar estados visitados recientemente.
- Para implementarla, es necesario experimentar para encontrar los valores más adecuados para los parámetros.
- La ascensión de gradiente se utiliza para buscar un máximo local en problemas con espacios de estados continuos.
- En cada iteración, se mueve en la dirección del ascenso más empinado, que está dada por el vector gradiente en ese punto.
- Es fundamental elegir un tamaño de paso adecuado.

➡️ Próximamente

Hasta ahora, los algoritmos de búsqueda vistos tratan a los estados como **cajas negras**. Simplemente consultan si son objetivo y aplican sobre ellos funciones heurísticas.

Vamos a **enriquecer la representación de los estados** para introducir una amplia familia de nuevos métodos de búsqueda y para generar una comprensión más profunda de la estructura de los problemas y de su complejidad.